High Performance Programming

”Parallel Program Paradigm and Design Process”

Ramoni Adeogun

AALBORG
UNIVERSITET

Content

1. Parallelizability of Problems
2. Design Paradigms
3. Parallel Algorithm Design Process
4. Characteristic of Tasks and Interactions
5. Summary and Exercise

Parallelizability of Problems

Parallel Program Development Process

Problem formulation

Algorithm design

Implementation

Optimization

Design of Parallel Programs

- Understand the problem (and algorithm)
  - Understand the problem to be solved in parallel
  - If starting with a sequential algorithm, its understanding is important
- Determine whether the problem is (actually) **parallelizable** → avoid time wastage
- Identify and focus on the **hotspots** → where is the most work done?
- Identification of **bottlenecks** → any disproportionately slow parts? e.g., input/output
- Identify **parallelism inhibitors** e.g., data dependence

Parallelizability of Problems

- Two extreme cases:
  - **embarrassingly/perfectly parallel problems** → little or no effort is needed to separate the problem into a number of parallel tasks
    - often the case where there is little or **no dependency** or **need for communication** between those parallel tasks, or for results between them
    - Example:
      - Monte Carlo simulations
      - Image processing
  - Inherently serial problems/**non-parallelizable** problems → cannot be split up into parallel portions
    - Calculate the Fibonacci series using a simple recursive approach:
- But what is in between these extreme cases?

Parallelizability of Problems

- Embarrassingly Parallel:
  - Perfectly parallel, no dependencies and tasks can run entirely independently.

- **Regular Parallel:**
  - Very high parallelism, but a few tasks may require some dependency or coordination.

- **Slightly Parallel:**
  - Some parallelism possible, but a significant portion of the task must be done sequentially.

- **Difficult to Parallelize:**
  - Requires complex synchronization and task management.

- Non-Parallelizable:
  - Cannot be parallelized due to inherent dependencies between tasks

Key Considerations for Parallelizability

- **Task Independence:**
  - How **independent** are the tasks from one another? → More independence means higher parallelizability.

- **Dependencies:**
  - How interdependent are the tasks? → More dependencies mean harder or less parallelizable.

- **Synchronization Costs:**
  - Does the problem require **frequent communication or coordination** between parallel tasks? → High synchronization costs reduce the effectiveness of parallelism.

- **Granularity:**
  - The size of the subproblems that can be parallelized → Finer granularity can increase parallelism, but it also increases overhead.

Design Paradigms

Binary Tree Paradigm

- A binary tree with n nodes is of height $\log_2(n)$
  - Suppose there are n data items, corresponding to the n leaf vertices of a **complete binary tree**
  - Process them in **parallel and get partial results** at the non-leaf nodes
  - Proceed bottom up, and reach the root in $\log_2(n)$ time

Example: **Sum of n numbers**

- It takes O(n) time for a single processor to compute the sum
- Assume $n = 2^d = 8$ and $p = \frac{n}{2} = 4$

- Suppose that the data to be added is

Item	A(1)	A(2)	A(3)	A(4)	A(5)	A(6)	A(7)	A(8)
Value	9	15	8	10	22	7	4	6

- Sketch the binary tree for a parallel program using the Binary Tree Paradigm

Binary Tree Paradigm

Algorithm **parallelSUM**

Input: Array $A[1, \dots, n]$, where $n = 2^d$

Output: The sum of the values in the array stored in $A[1]$

1: // computes sum of $A[1], A[2], \dots, A[n]$
2: $z = \frac{n}{2}$
3: **while** $z > 0$ **do**
4: **for** $id = 1 : z$ **do in parallel**
5: $A[id] = A[2 \times id - 1] + A[2 \times id]$
6: **End parallel**
7: $z = \left\lceil \frac{z}{2} \right\rceil$
8: **return** $A[1]$

Time complexity: $O(\log_2(n))$

Using $O(n)$ processors on an EREW PRAM architecture

Stage 1:

P_1 do $A(1) \leftarrow A(1) + A(2) = 24$
P_2 do $A(2) \leftarrow A(3) + A(4) = 18$
P_3 do $A(3) \leftarrow A(5) + A(6) = 29$
P_4 do $A(4) \leftarrow A(7) + A(8) = 10$

Stage 2:

P_1 do $A(1) \leftarrow A(1) + A(2) = 42$
P_2 do $A(2) \leftarrow A(3) + A(4) = 39$

Stage 3:

P_1 do $A(1) \leftarrow A(1) + A(2) = 81$

Growing By Doubling

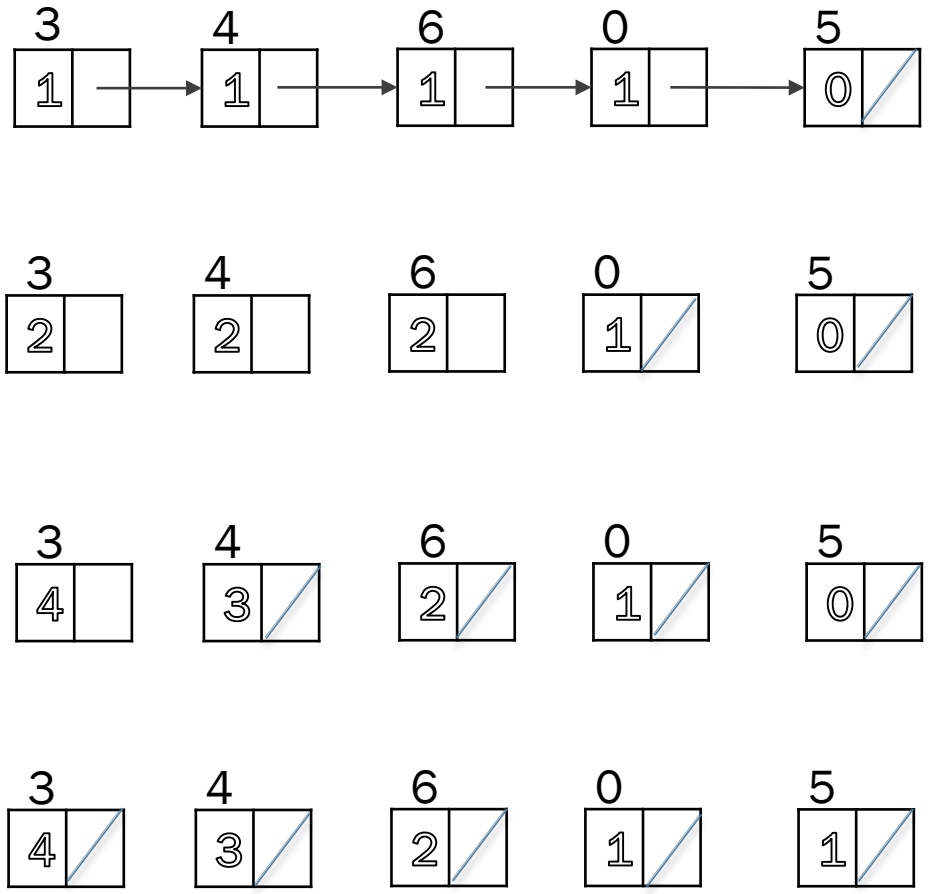
Concept:

- Doubling strategy is a common parallel programming technique where the **problem size or work units double** at each step.
  - At the initial step, each processor covers a single entity and waits for consolidation
  - At every step the number of entities covered is double the entities covered at the previous step: 2, 4, 8, etc.

- Example: List Ranking
  - Given a **linked list**, the goal is to assign a rank to each node, indicating its distance from the head.
  - Used in parallel computing for **graph algorithms**, **parallel prefix computation**, and **Euler tour** techniques.

List Ranking

- Sequential
  - Traverse each node one by one → $O(n)$ time
- Parallel (Growing by Doubling)
  - Nodes update their ranks in parallel → $O(\log_2(n))$ time
    - Pointer jumping steps
  - Update rule:
    - $Next(X) = Next\big(Next(X)\big)$
    - $Rank(X) = Rank(X) + Rank\big(Next(X)\big)$

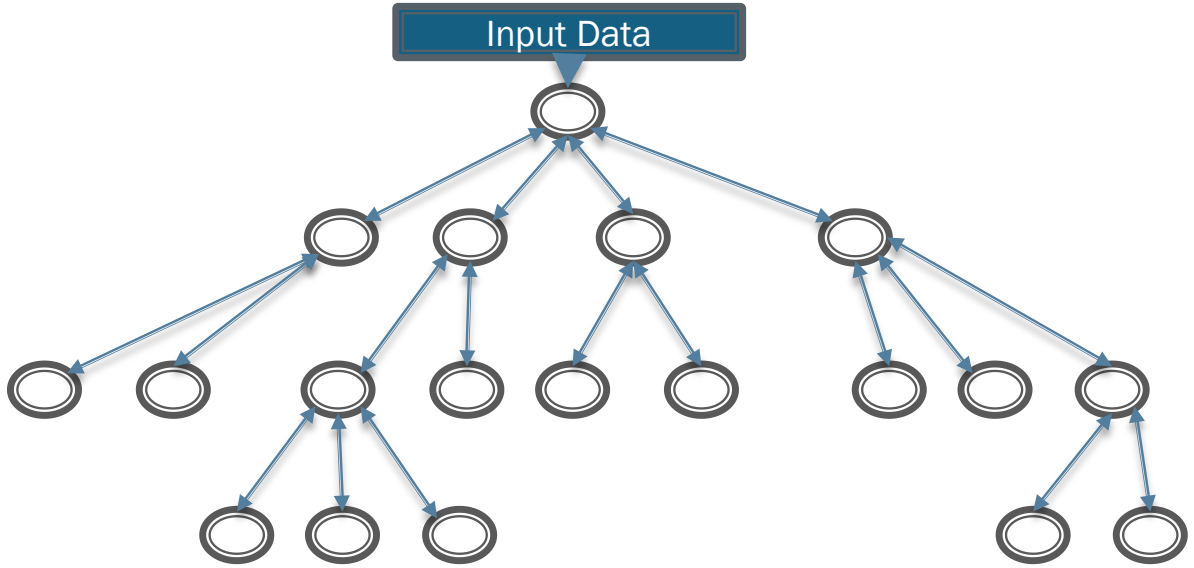


Divide and Conquer

- Divide and Conquer technique: If there is a direct solution to a given problem then it is applied directly. Otherwise the problem is repeated partitioned into subproblems.

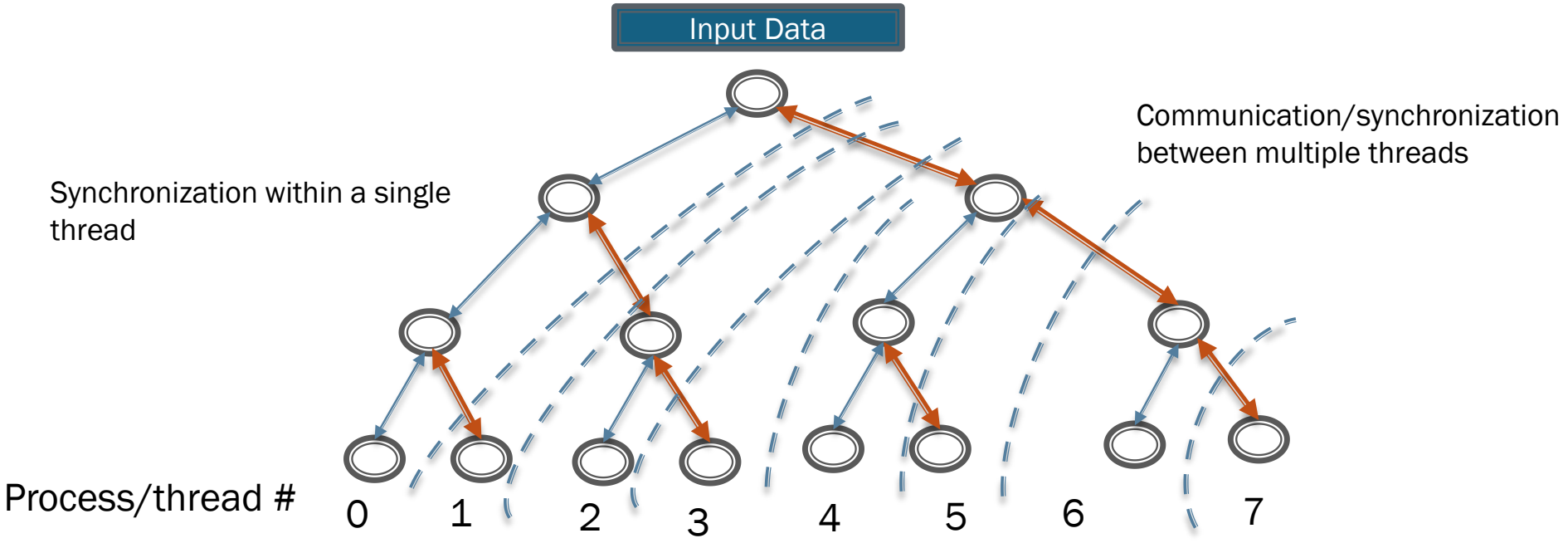
- A divide-and-conquer algorithm can be represented as a binary tree:
  - Characteristics:
    - Node degrees
    - Tree Depth

What makes good characteristics of a binary for divide-and-conquer?



An imbalanced tree illustration of divide-&-conquer processing

Parallelization of Divide-and-conquer Tree



- A divide-and-conquer application can be very dynamic in terms of:
  - Dependency of subtree generation on previously computed results
  - Varying time for processing nodes, especially leaves of the tree.

- A dynamic approach is neccessary for load balancing.

Parallel Algorithm Design Process

Parallel Algorithm Design Principle

- A given programming problem may have multiple parallel algorithmic solutions
  - Best solution may not be the one suggested by existing sequential algorithms

- General design process:
  - **focus on machine-independent issues** (concurrency, parallelism, etc)
  - Machine dependent considerations are left for later part of the design process

- Four stage design process:
  - **Decomposition** → Communication → Agglomeration/Clustering → Mapping

Decomposition (or Partitioning)

- Decompose the problem into tasks that can be executed concurrently
  - show opportunities for parallel execution
  - obtain a fine-grained decomposition of the problems → large number of small tasks
- A good partition divides into small pieces both the computation associated with a problem and the data on which this computation operates
  - A given problem may be decomposed into tasks in many ways.
- **Two possible approaches:**

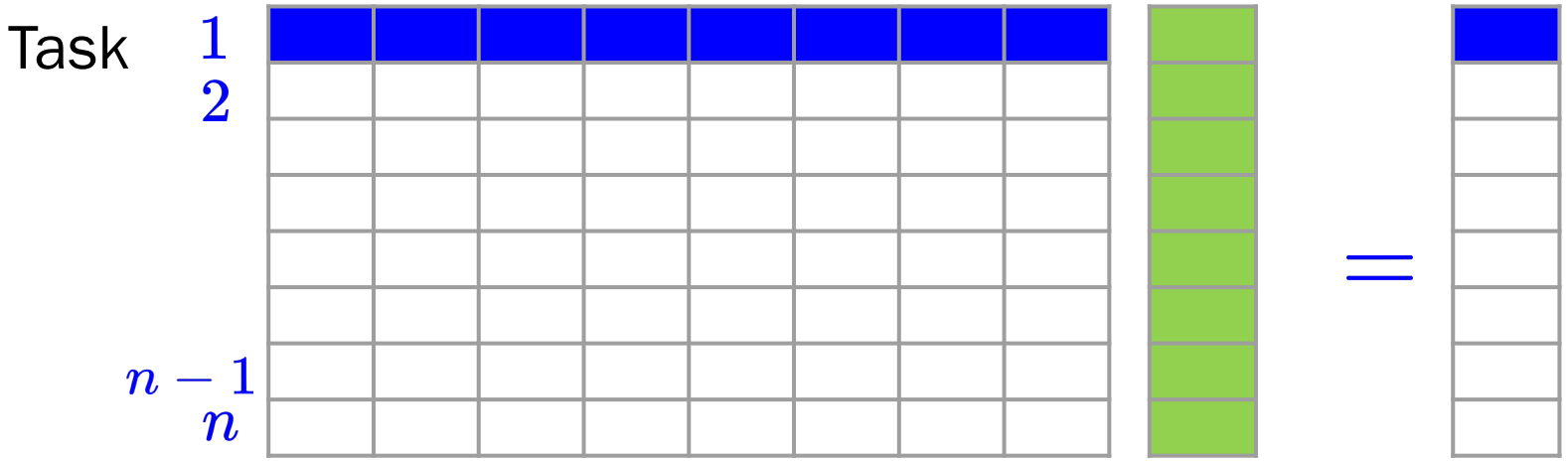
  1. **Domain decomposition:** data partitioning first and then associated computations
  2. **Function decomposition:** start with portioning of the computations and then data

A decomposition can be illustrated in the form of a **directed graph**

- nodes corresponding to tasks
- edges indicating that the result of one task is required for processing the next.
- Such a graph is called a **task dependency graph or computation graph or ..... .**

Example: Dense Matrix-vector Multiplication

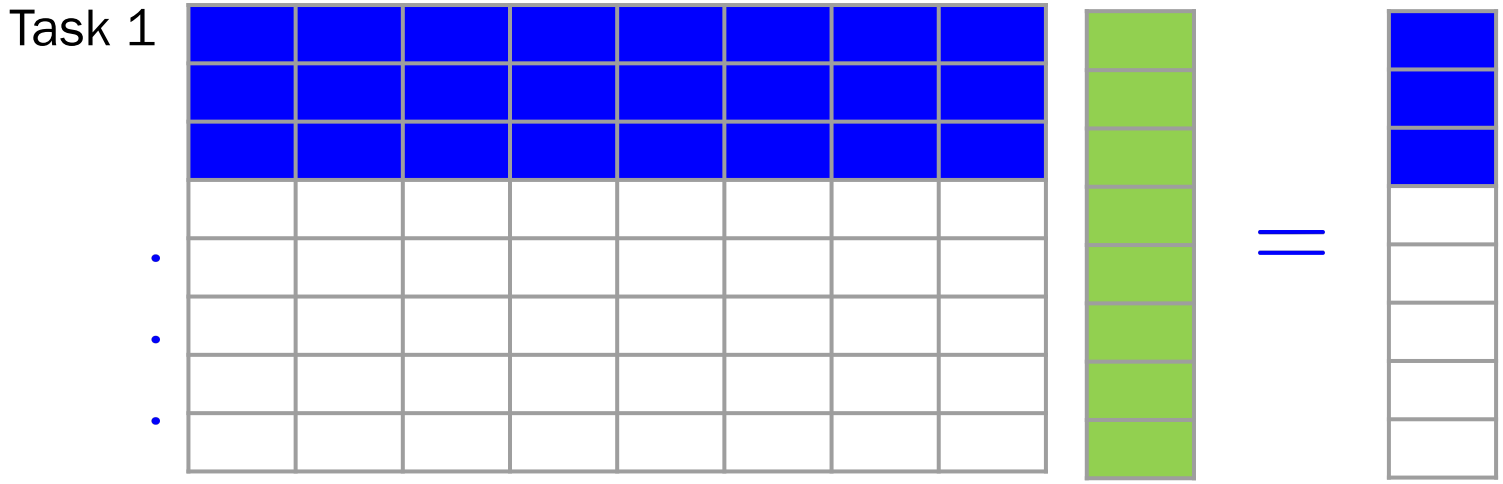
- Consider the matrix vector multiplication: $y = A \times b$



- Computation of each element of output vector y is independent of other elements.
  - a dense matrix-vector product can be decomposed into n tasks.
- Observations:
  - While tasks share data (i.e., b), they do not have any control dependencies - i.e., no task needs to wait for the (partial) completion of any other.
  - All tasks are of the same size in terms of number of operations.
  - Is this the maximum number of tasks we could decompose this problem into?

Granularity of Task Decompositions

- The number of tasks into which a problem is decomposed determines its **granularity**.
- Decomposition into:
  - a large number of tasks results in **fine-grained decomposition**
  - a small number of tasks results in a **coarse-grained decomposition**.



A coarse grained counterpart to the dense matrix-vector product example.

- Each task in this example corresponds to the computation of three elements of the result vector.

Data (Domain) Decomposition: Example

- Consider the problem of multiplying two $n \; x \; n$ matrices A and B to yield matrix C.
- The output matrix C can be partitioned into four tasks as follows (or even 8):

$$\begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix} \cdot \begin{pmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{pmatrix} \rightarrow \begin{pmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{pmatrix}$$

$$C_{1,1} = A_{1,1}B_{1,1} + A_{1,2}B_{2,1}$$

$$C_{1,2} = A_{1,1}B_{1,2} + A_{1,2}B_{2,2}$$

$$C_{2,1} = A_{2,1}B_{1,1} + A_{2,2}B_{2,1}$$

$$C_{2,2} = A_{2,1}B_{1,2} + A_{2,2}B_{2,2}$$

Decomposition I	Decomposition II
Task 1: $C_{1,1} = A_{1,1} B_{1,1}$	Task 1: $C_{1,1} = A_{1,1} B_{1,1}$
Task 2: $C_{1,1} = C_{1,1} + A_{1,2} B_{2,1}$	Task 2: $C_{1,1} = C_{1,1} + A_{1,2} B_{2,1}$
Task 3: $C_{1,2} = A_{1,1} B_{1,2}$	Task 3: $C_{1,2} = A_{1,2} B_{2,2}$
Task 4: $C_{1,2} = C_{1,2} + A_{1,2} B_{2,2}$	Task 4: $C_{1,2} = C_{1,2} + A_{1,1} B_{1,2}$
Task 5: $C_{2,1} = A_{2,1} B_{1,1}$	Task 5: $C_{2,1} = A_{2,2} B_{2,1}$
Task 6: $C_{2,1} = C_{2,1} + A_{2,2} B_{2,1}$	Task 6: $C_{2,1} = C_{2,1} + A_{2,1} B_{1,1}$
Task 7: $C_{2,2} = A_{2,1} B_{1,2}$	Task 7: $C_{2,2} = A_{2,1} B_{1,2}$
Task 8: $C_{2,2} = C_{2,2} + A_{2,2} B_{2,2}$	Task 8: $C_{2,2} = C_{2,2} + A_{2,2} B_{2,2}$

- Unique decomposition of tasks is not guaranteed.

Communication

- Tasks from problem decomposition are intended to be executed concurrently
  - concurrency does not generally imply independence
  - operations performed on one task typically require data associated with another task(s)

- The communication phase of the design involve defining how data is transferred between multiple concurrent task executions

- Communication in parallel processing can be seen as a two stage process:
  - specify communication channels between tasks (considering topology)
  - define the type of messages to be sent on the channels

Goal: optimize performance by distributing communication operations over many tasks and by organizing communication operations in a way that permits concurrent execution.

- minimize number of channels and communication requirements.

Distribution of Communication and Computation

- We return to the compute sum algorithm for finding the summation of N numbers, i.e.,

$$S = \sum_{n=1}^{N} X_n$$

- We can distribute the computation into N tasks such that the nth task compute

$$S_n = X_n + S_{n-1}$$

- The compute sum algorithm
  - distributes the N-1 communication and additions
  - permits concurrent execution **only** if multiple operations are to be performed

How do we uncover concurrency?

- To solve a complex problem, we use divide and conquer
  - recursively partition the problem into two or more subproblems
  - solve subproblems and combine solutions

- For the summation problem
  - Using the identity: $N = 2^n$, we can decompose the problem into two:

$$\sum_{i=0}^{2^n-1} X_i = \sum_{i=0}^{2^{n-1}-1} X_i + \sum_{i=2^{n-1}-1}^{2^n-1} X_i$$

Binary Tree Representation

How does the divide and conquer algorithm compare with the linear array summation in the previous slide?

How do we access communication design for a parallel algorithm?

- **Do all tasks perform about the same number of communication operations?**
  - Unbalanced communication requirements suggest a non-scalable construct.
  - Revisit your design to see whether communication operations can be distributed more equitably.

- **Does each task communicate only with a small number of neighbors?**
  - If each task must communicate with many other tasks, evaluate the possibility of formulating this global communication in terms of a local communication structure.

- **Are communication operations able to proceed concurrently?** If not, your algorithm is likely to be inefficient and non-scalable.

- **Is the computation associated with different tasks able to proceed concurrently**? If not, your algorithm is likely to be inefficient and non-scalable. Consider whether you can reorder communication and computation operations.

Agglomeration/Clustering

- So far:
  - Decomposition: Partition computation into a set of tasks
  - Communication: design channels to provide data to the tasks

- But the design is still rather abstract → no specialized mechanism for execution on any parallel computer.

- Shifting a bit from abstraction to more concreteness → obtaining parallel algorithms to run on specific class or classes of parallel computers
  - revisit decomposition and communication design choices
  - consider combining tasks to provide a reduced number of larger tasks
  - determine whether algorithm may benefit from replication of data and/or computation

Processes and Mapping

- In general, the number of tasks in a decomposition exceeds the number of processing elements.
  - a parallel algorithm must also provide a mapping of tasks to processes.

- We refer to the mapping as being from **tasks to processes**, as opposed to processors.
  - typical programming APIs do not allow easy binding of tasks to physical processors.
  - we aggregate tasks into processes and rely on the system to map these processes to physical processors.
  - processes here simply refer to a **collection of tasks and associated data.**

- Appropriate mapping of tasks to processes is critical to **parallel performance**
  - Mappings are determined by both the **task dependency** and **task interaction** graphs.
  - Task dependency graphs can be used to ensure that work is equally spread across all processes at any point (**minimum idling and optimal load balance**).

Processes and Mapping

- Task interaction graphs can be used to make sure that processes need minimum interaction with other processes (minimum communication).

- An appropriate mapping must minimize parallel execution time by:
  - Mapping independent tasks to different processes.
  - Assigning tasks on **critical path** to processes as soon as they become available.

- Minimizing interaction between processes by mapping tasks with dense interactions to the same process.

Note: These criteria often conflict each other. For example, a decomposition into one task (or no decomposition at all) minimizes interaction but does not result in a speedup at all!

Can you think of other such conflicting cases?

Characteristic of Tasks and Interactions

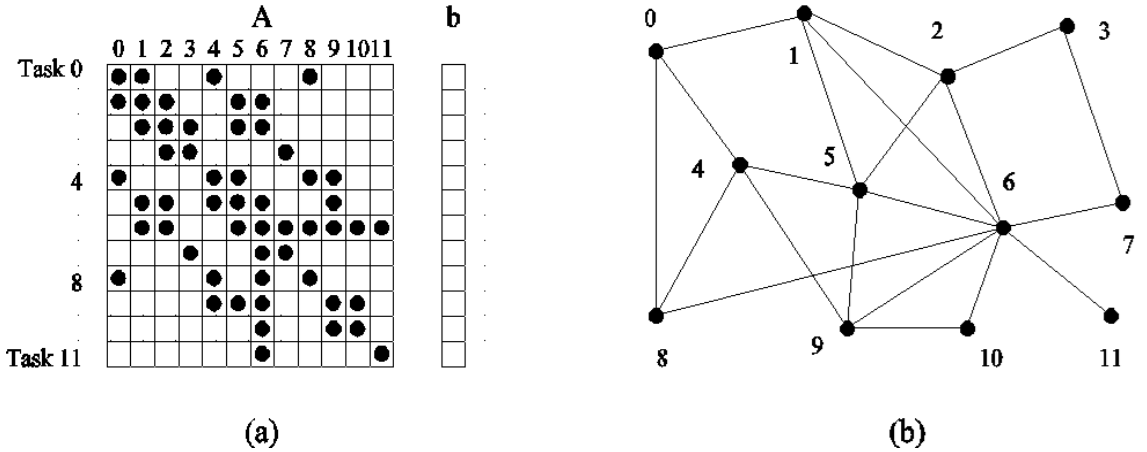
Task Interaction Graphs

- Subtasks generally exchange data with others in a decomposition.
  - The graph of tasks (nodes) and their interactions/data exchange (edges) is referred to as a task interaction graph.
  - task interaction graphs represent data dependencies, whereas task dependency graphs represent control dependencies.

Example: Consider the problem of multiplying a sparse matrix A with a vector b.

  - the computation of each element of the result vector can be viewed as an independent task.
  - only non-zero elements of matrix A participate in the computation.

- If, for memory optimality, we also partition b across tasks, then the task interaction graph of the computation is identical to the graph of the matrix A (the graph for which A represents the adjacency structure).



Characteristics of Task Interactions

- Tasks may communicate with each other in various ways:
  - Static interactions: The tasks and their interactions are known a-priori.
    - These are relatively simpler to code into programs.
  - Dynamic interactions: The timing or interacting tasks cannot be determined a-priori.
    - These interactions are harder to code, especially, as we shall see, using message passing APIs.
  - Regular interactions: There is a definite pattern (in the graph sense) to the interactions.
    - These patterns can be exploited for efficient implementation.
  - Irregular interactions: Interactions lack well-defined topologies.
- Interactions may be read-only or read-write.
  - In read-only interactions, tasks just read data items associated with other tasks.
  - In read-write interactions tasks read, as well as modify data items associated with other tasks.
- Interactions may be one-way or two-way.
  - A one-way interaction can be initiated and accomplished by one of the two interacting tasks.
  - A two-way interaction requires participation from both tasks involved in an interaction.
  - One way interactions are somewhat harder to code in message passing APIs.

Degree of Concurrency

- The number of tasks that can be executed in parallel is the degree of concurrency of a decomposition.

- Since the number of tasks that can be executed in parallel may change over program execution,
  - the maximum degree of concurrency is the maximum number of such tasks at any point during execution.
  - What is the maximum degree of concurrency of the summation example?

- The **average degree of concurrency** is the average number of tasks that can be processed in parallel over the execution of the program.

- The degree of concurrency increases as the decomposition becomes finer in granularity and vice versa.

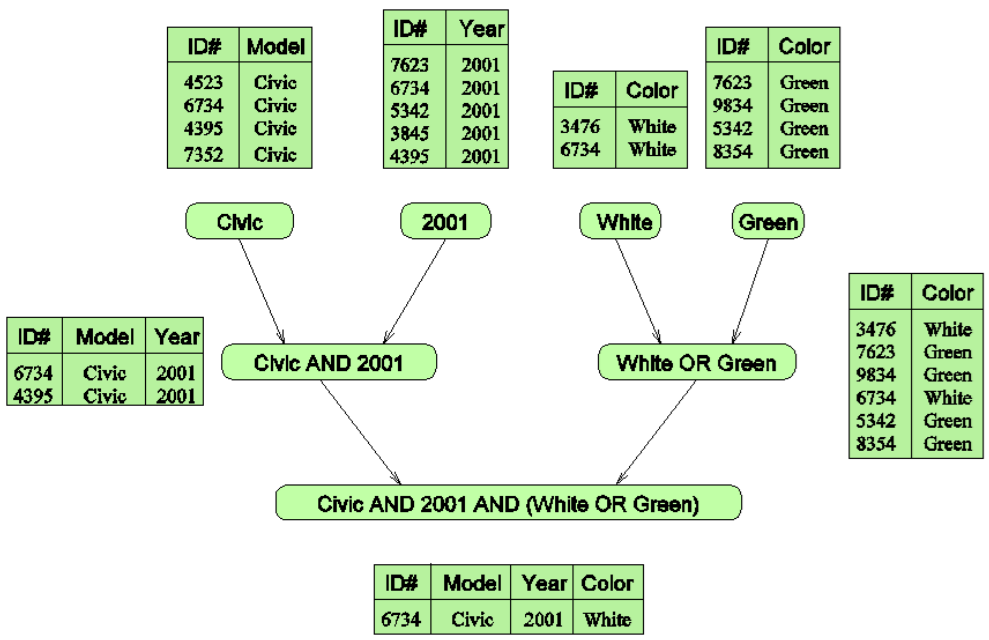
Example: Database Query Processing

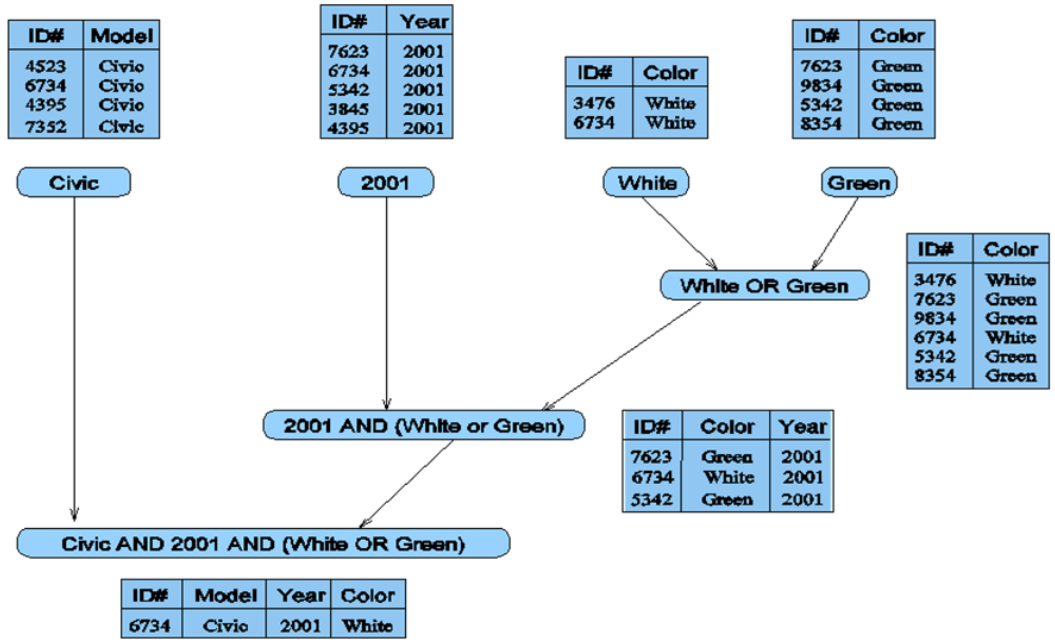
Consider the execution of the query: *MODEL = ``CIVIC'' AND YEAR = 2001 AND (COLOR = ``GREEN'' OR COLOR = ``WHITE)* on the following database:

ID#	Model	Year	Color	Dealer	Price
4523	Civic	2002	Blue	MN	$18,000
3476	Corolla	1999	White	IL	$15,000
7623	Camry	2001	Green	NY	$21,000
9834	Prius	2001	Green	CA	$18,000
6734	Civic	2001	White	OR	$17,000
5342	Altima	2001	Green	FL	$19,000
3845	Maxima	2001	Blue	NY	$22,000
8354	Accord	2000	Green	VT	$18,000
4395	Civic	2001	Red	CA	$17,000
7352	Civic	2002	Red	WA	$18,000

Example: Database Query Processing

- The execution of the query can be divided into subtasks in various ways. Each task can be thought of as generating an intermediate table of entries that satisfy a particular clause.





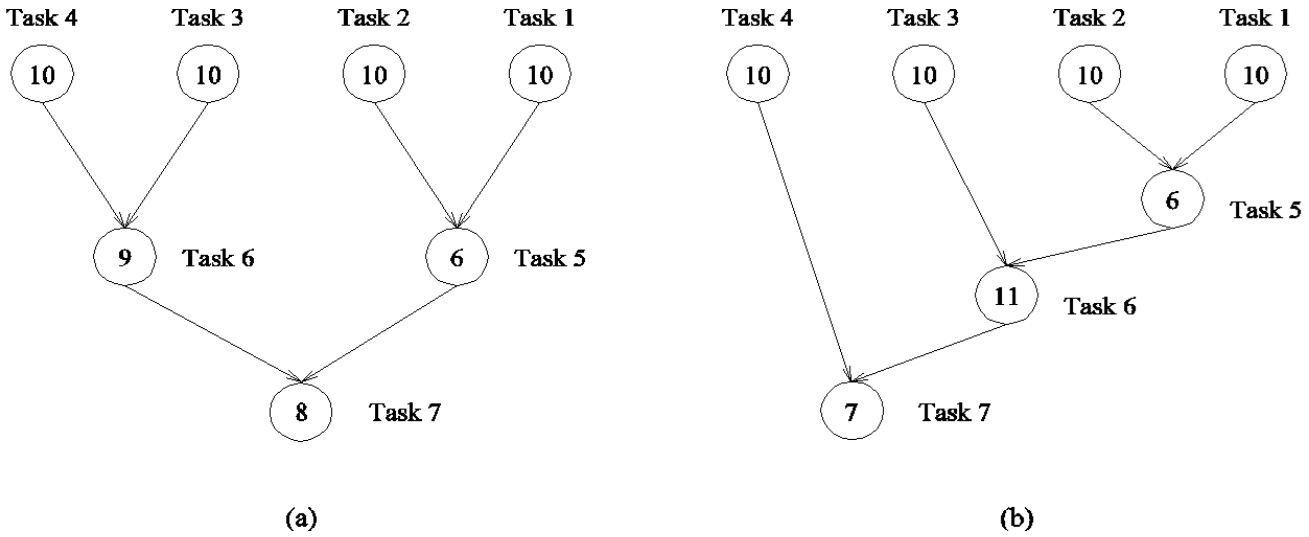
- Two possible decomposition of the given query into a number of tasks. Edges in this graphs denote that the output of one task is needed to accomplish the next.

Critical Path Length

A directed path in the task dependency graph represents a sequence of tasks that must be processed one after the other.

- The longest such path determines the shortest time in which the program can be executed in parallel.
- The length of the longest path in a task dependency graph is called the **critical path length**.

Consider the task dependency graphs of the two database query decompositions:



Property	a	b
Max deg. of conc.		
Av. deg of conc.		
Critical path length		

How do we map these task to processes (assuming a maximum of 4)?

Limits on Parallel Performance

- It would appear that the parallel time can be made arbitrarily small by making the decomposition finer in granularity.
- But there is an inherent bound on how fine the granularity of a computation can be.

- For example, in the case of multiplying a dense matrix with a vector, there can be no more than (n^2) concurrent tasks.

- Concurrent tasks may also have to exchange data with other tasks → communication overhead.

- The tradeoff between the **granularity of a decomposition** and associated **overheads** often determines performance bounds.

Summary and Exercise

Summary

- Not all problems are parallelizable
  - Pay careful attention to understanding the problem and its parallelizability before venturing into algorithm design
- Several paradigms exist for analysis and design of parallel algorithms
  - Binary tree, growing by doubling, parallel divide-and-conquer, partitioning
    - Use of each paradigm depends on the type of problem and its parallelizability
- Design of a parallel algorithm for any given problem can be divided into 4 stages:
  - Decomposition/partitioning → communication design → agglomeration → mapping
- Task dependency and task interaction graphs are useful for analysing communication and the mapping of tasks to processes

- Even for parallelizable problems, there is a limit on the granularity of decomposition
  - Consider how fine or grained the granularity should be for best performance.

Summary

- Two sometimes conflicting useful strategies for mapping
  - place tasks that can execute concurrently on different processors to enhance concurrency.
  - place tasks that communicate frequently on the same processor, to increase locality.

- Typical algorithms rely on domain decomposition
  - Efficient in cases featuring equal size tasks and structured communication
  - May be difficult on applications with varied work per task and/or unstructured communication
  - Use load balancing to identify efficient agglomeration and mapping techniques

That’s it for today!